显存受限下的部分驻留 KV Cache 解码设计

面试题设计答卷 · 单卡、共享显存池、按需加载 · 2026-11-06

1. 设计结论与问题边界

核心方案：用 host 保存完整逻辑历史，用同一个 GPU 池同时承载驻留页和有界 hot 区；每步只把选中且缺失的 KV rows 装入 hot。准入保证这一步的工作集有落点，而不要求整个请求的 KV 重新驻留。

这直接回答题目的三个问题：

问题	本方案的决策
页面在哪里，怎样卸载和临时加载?	逻辑页有稳定 host 地址和可撤销的 GPU 映射；实际复用才撤销驻留映射；临时加载用独立 hot 内容表管理。
共享池中驱逐谁、预留多少?	保证每层 hot 容量覆盖单步选择并集，另保留两页 tail；优先使用空 block，再回收镜像完成、无人消费的历史页。
如何减少加载等待?	提前镜像、只加载 misses、跨头去重、设备批量规划；用完成事件保护搬运与读取。进一步可将已就绪部分 attention 与 miss 加载重叠。

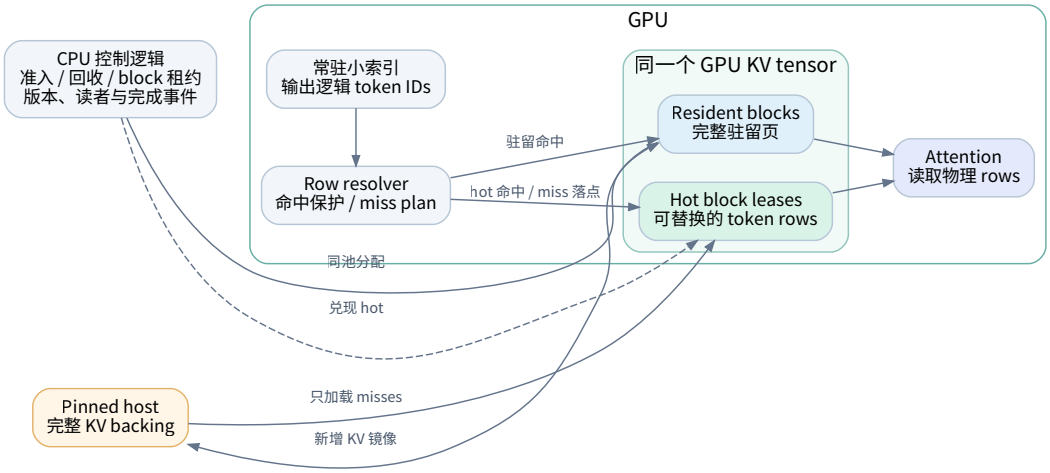


图1：resident 与 hot 是同一个 GPU tensor 的不同 block 租约；host backing 是另一个、容量受约束的 pinned-memory 池。小索引和元数据另计 GPU 预算。

约束与假设。 一个控制器管理单卡上的多个请求；prefill 提供初始 KV，之后只做增量 decode。逻辑选择器提供本步 token IDs，每个请求/层的跨头并集不超过已声明上界 U。历史 KV 写入后不再修改，只有当前 tail 追加。GPU 操作最终完成，消费者按接口释放租约。

保证与不保证。 已准入请求可以在历史页被回收后继续 decode，无需恢复完整 KV 或重做 prefill；新请求仍可能等待准入。保证针对已声明的选择集合，不保证近似稀疏选择与 dense attention 的模型质量相同。KV pool 的上限也不包含模型权重、索引、激活和 prefill workspace。

全文主线是：**地址与状态 → 容量承诺 → 回收与替换 → 一步 decode → 异步优化 → 正确性与验证**。读者无需先阅读任何项目文档。

2. 数据模型：页管理所有权，row 管理选择和搬运

每页包含 P 个 tokens。一个 KV row 指一个 token 在该层的 K/V，包含全部 KV heads；一个 GPU block 是池中可分配的物理页槽。页身份与物理 block 不能混为一谈。

```
logical_page = (request_id, request_generation, layer, page_index)
GPU / host tensor layout = [blocks, 2, P, kv_heads, head_dim]
physical_row_id = block_id * P + offset
```

Physical row ID 是地址编码；kernel 据此分别定位 K/V 两个平面，不把它当作五维 tensor 的任意连续切片。

元数据	内容	更新时机
Page record	host block、GPU block 或空值、有效行数、sealed、消费者计数	首次写入、追加、实际复用
Mirror versions	内容版本、已提交镜像版本、已完成镜像版本及事件	写入、D2H 提交、完成发布
Source / resident tables	逻辑页 → host block / GPU block；GPU 不驻留时为 -1	页创建、绑定、撤销映射
Hot block table	本请求/层的 hot blocks 列表，可不连续	整组分配和请求退出
Hot tags / LRU	hot slot → host physical row；替换顺序	本轮命中保护、miss 分配
Block owner / generation / fence	当前所有者、复用代次、旧访问完成事件	物理 block 绑定和释放

CPU 管理容量和生命周期，设备表解析地址；hot tags 识别 host rows。跨头选择先去重，再保留各 head 的 mask，同一 row 只搬一次且不重复计入 softmax。Host backing 在请求退休事件完成前不得复用。

读一个 token t 的规则： 先计算 `page, offset = divmod(t, P)`。若 resident table 有效，直接使用 resident row；否则用 `source[page]*P+offset` 查 hot tag，未命中才安排加载。

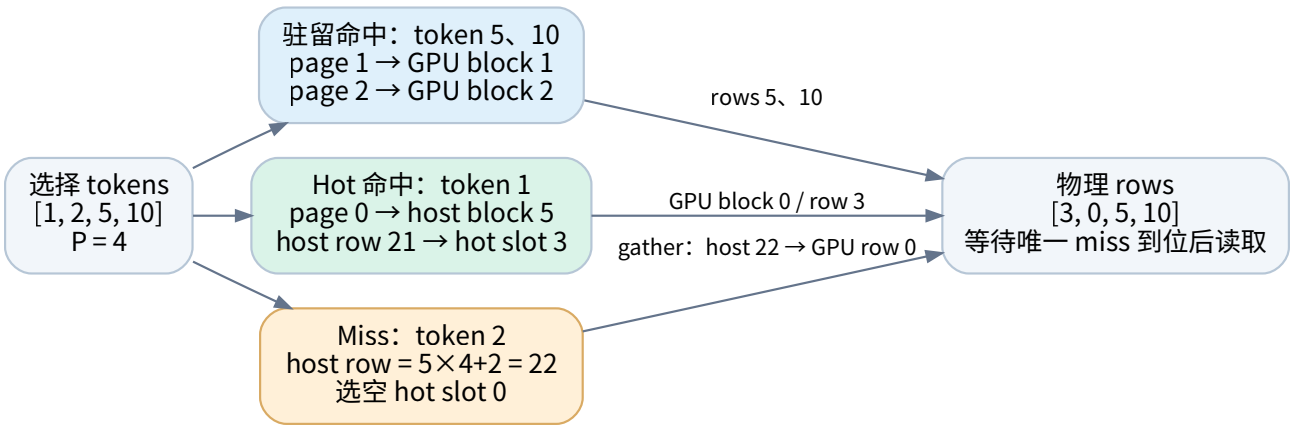


图2是手算示例：P=4、U=H=4；一个 hot block 加两个 resident tail blocks，总共三个 GPU blocks。它用于说明寻址，不是性能测量。

3. 页面状态：卸载不丢历史，临时加载不恢复整页

新 KV 优先写入 GPU，并异步镜像新增 rows 到 host。只在对应版本 D2H **完成**后发布 clean；“copy 已提交”和“页已经写满”都不能代替完成条件。

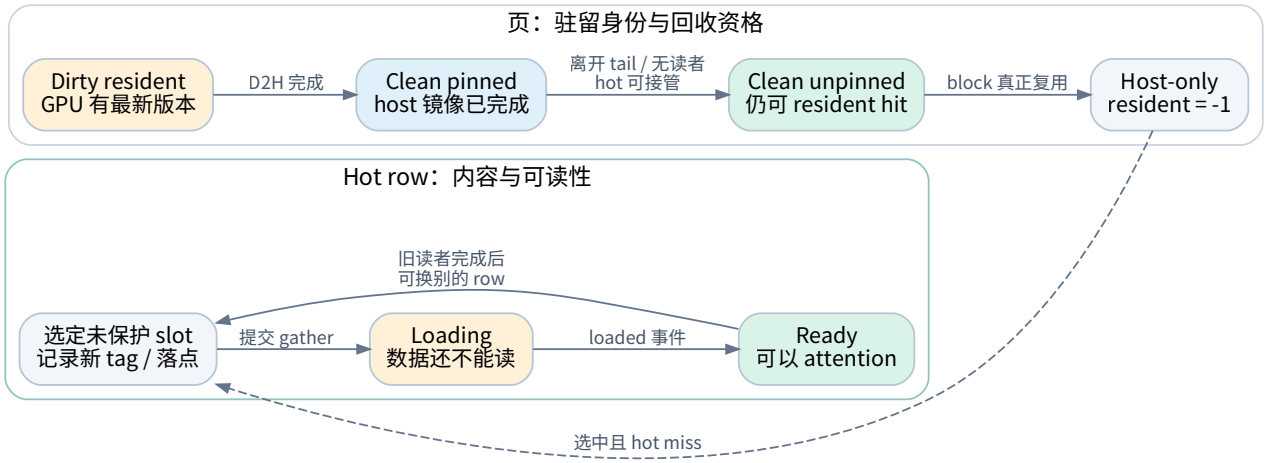


图3：clean-unpinned 仍保留驻留映射；host-only 页的部分 rows 可以同时存在于 hot 中。Hot 内容就绪还取决于 gather 完成事件。

状态转换规则：

1. **写入后是 dirty resident。** 内容版本递增，提交增量 D2H；后续写入同一 tail 前要尊重上一次镜像和读者事件。
2. **镜像完成后成为 clean pinned。** host 已有最新副本，但是否可回收还取决于 tail、消费者和 hot 接管条件。
3. **合格历史页成为 clean unpinned。** 页已 sealed，位于最后两页之外，无消费者租约，hot 已就绪或参与一次完整的 hot 分配事务。此时 block 可回收，但数据与 GPU 映射仍有效。
4. **实际复用时才变为 host-only。** 回调撤销旧 resident entry；保留 source entry；新 owner 绑定时 generation 增加。新写入必须等待旧 block fence。
5. **再次选中时按 row 加载。** Hot miss 选落点并提交 gather；attention 等 ready 才读。逻辑页仍是 host-only，hot slots 以后可以缓存别的页中的 rows。

```
def make_reclaimable(page):
    if not hot_ready(page.group): return
    if not sealed_history_without_consumers(page): return
    if not latest_host_copy_completed(page): return
    reclaimable.enqueue_once(page.block)   # 保留 resident entry

def reuse_block(block):
    old = owner(block)
    assert safe_to_reclaim(old)
    resident[old.page] = -1                # 真正复用时才撤销
    fence = join_events(old.write, old.mirror, old.last_read)
    return block, fence                    # 新写者必须等待 fence
```

4. 容量承诺：保证当前一步，而非完整历史

令 U 为一个请求/层单步选中 token 的最大跨头并集，H 为 hot token slots，P 为页大小，L 为层数。每请求保留：

```
H >= U
hot_blocks_per_layer = ceil(H / P)
device_credit_per_request = L * (hot_blocks_per_layer + 2)
所有已准入请求的 device credits 之和 <= GPU KV pool blocks
```

另外，按最大请求长度预留 host backing credit 和 GPU 索引字节。三个预算都满足才准入。**Credit 是未来容量承诺，不是立即分配 hot blocks**；hot 尚未兑现时，resident history 可借用这些物理空间。

以 `P=16`、`U=32`、`L=2` 为例：H=32 时需 8 blocks/request；H=64 时需 12。一个 block 的字节数为 `2*P*kv_heads*head_dim*dtype_bytes`。默认 H=2U 可保留更多跨步内容，但不是正确性所需的最低值，也不是最优性能保证。

为什么 H≥U 足够?

设本步有 r 个 resident hits、h 个 hot hits、m 个 misses。因为 `r+h+m<=U`，保护 h 个 hot hits 后：

```
可替换 hot slots = H-h >= U-h >= m
```

因此本步缺失 rows 一定有落点。替换旧 hot 内容不需要写回：它只是不可变历史 KV 的副本，最新 host backing 已存在。Hot region 内没有新的可丢失的模型状态。

为什么再留两页 tail?

当前追加页和它前一页固定驻留。跨页时，保护区向前移动，更老的页才进入候选集合；该页还必须 sealed、镜像完成、无人消费。追加页号在分配前就参与 tail 判定，避免先要求第三个 tail block 才允许旧页让位。

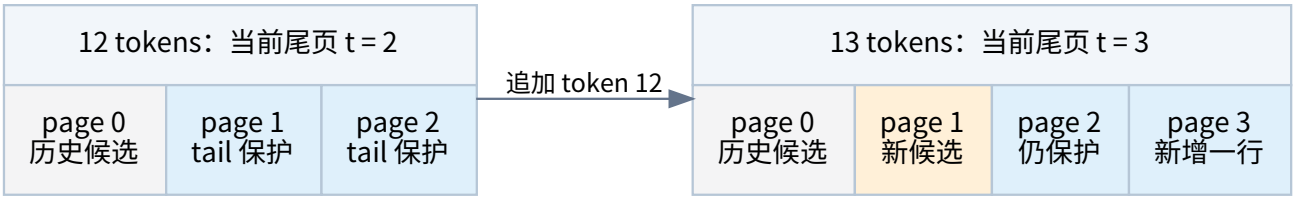


图4：P=4。跨页后 page 1 仅获得“候选资格”，并不代表它可以越过镜像完成或消费者保护立即被覆盖。

这里的进展保证有明确前提：selection 不超过 U、租约正常释放、GPU 工作最终完成、host/index 预算已经兑现。它不承诺任意小显存都能运行，也不承诺预留了完整 prefill workspace。若 selection 没有上界，就不能用固定大小 hot 区保证一次装下，需要另行设计分窗 attention；本方案不把该能力当作默认前提。

5. 共享池分配：先选安全候选，再整体兑现 hot

回收顺序是：空 block → 已可回收且无人消费的 clean block → 本层可安全让出的历史页。 不覆盖最后两页，不覆盖尚未形成最新 host 副本的页，不覆盖租约仍保护的内容。

常规合格历史页按逻辑页号从大到小加入可回收队列，实际复用按队列取。这保留了较早前缀的机会，但只是启发式，不是全局访问 LRU。用于初次 hot 分配的本层补足候选按 allocator 顺序选取。Hot 内部则使用独立的、请求/层局部的 row LRU。

何时申请 hot? 当可分配 blocks 低于水位，或本层 resident 页数达到窗口阈值时，设置持久 `hot_required`。在申请新 append block 前用 `available-1` 预判。可取默认水位 `max(每层 hot blocks, 总 blocks//10)`、窗口 64 页；二者都是可调策略参数，窗口不是 attention 截断范围。

```
def try_allocate_hot(group):
    if group.hot_allocated: return READY
    group.hot_required = True
    if group.has_active_consumer: return WAIT_READER
    need = ceil(H / P)
    candidates = empty_blocks_then_safe_reclaimable()
    candidates += safe_own_history_not_already_listed(group)
    if len(candidates) < need:
        return WAIT_RESOURCE              # 保留 demand 和所有旧映射
    chosen = candidates[:need]             # 先完整预检，再修改状态
    for block in chosen:
        block, fence = detach_if_owned(block)
        bind_hot(block, group, next_generation=True)
        transfer.wait_event(fence)
    publish_hot_table(group, chosen, ready=transfer.record_event())
    mark_eligible_history_reclaimable(group)
    return READY
```

这些元数据修改由单控制器串行执行。所谓“整体分配”是指不会因只获得一部分 hot blocks 就提前丢失旧映射；GPU 上的数据安全仍由 fence 保证。

例如需要四个 hot blocks，而当前只有两个可分配 blocks，就必须在本层找到另外两个安全历史 blocks；找不到则保留申请。不能先卸载一部分，再把请求留在没有可执行工作集的状态。

容量耗尽时如何继续? 若缺的只是一个合格历史页的 D2H 完成，等待这个具体事件后重试。生产调度器可将请求置为 `WAIT_MIRROR` 并执行其他请求；同步教学 runner 可等待该事件。两者都不能通过覆盖 dirty 页或重复 prefill 回避资源问题。

Hot slot 替换规则。 先对整个 selection 去重并找齐 hits，将所有选中 hot hits 保护起来，再给 misses 按局部 LRU 分配未保护 slots。不能边扫描边逐出，否则一个先出现的 miss 可能覆盖后面才遇到的 hit。Hot block 租约持续到请求结束；slot 内容替换不会再申请 block。

默认固定 H 使预算和进展条件清楚。若以后动态调整 H，必须连同全局 credit、block 租约、tags 和消费者 fence 一起变更；仅凭短期 hit rate 扩大 hot，可能反而挤掉更有价值的 resident history。

6. 一步 decode：从逻辑选择到可读的物理 rows

以下只保留 KV 路径。每请求/层最多一个 active lease；保守 pin 本层全部 resident 页以避免回读 GPU top-k，代价是暂时减少可回收容量。Selection 包含刚追加的新 KV。

```
def decode_layer(query, new_kv, group):
    append_to_resident_tail(group, new_kv)
    enqueue_incremental_host_mirror(group)  # 只镜像新增 rows
    selected = indexer(query, group.index_state)
    union, head_columns = deduplicate_per_head(selected)
    assert len(union) <= group.admitted_U
    lease = pin_residents_and_lease_hot(group)
    compute.wait_event(group.hot_ready)
    plan = resolve_on_device(union, source, resident, hot, tags, lru)
    resolved = compute.record_event()
    transfer.wait_event(resolved)
    transfer.wait_event(group.last_read)
    gather_host_rows(plan.misses)           # 前面的 host mirrors 已排在同流
    group.hot_ready = transfer.record_event()
    compute.wait_event(group.hot_ready)
    output = attention(query, plan.physical_rows, head_columns)
    group.last_read = compute.record_event()
    lease.release_after(group.last_read)
    return output
```

`plan` 给出每个选中 token 的 physical row，以及固定形状的 host-row/device-row miss plan；有效前缀之外填 -1。设备侧处理排序、去重、hit 保护和 victim 分配，不把 miss count 读回 CPU 再逐条安排搬运。

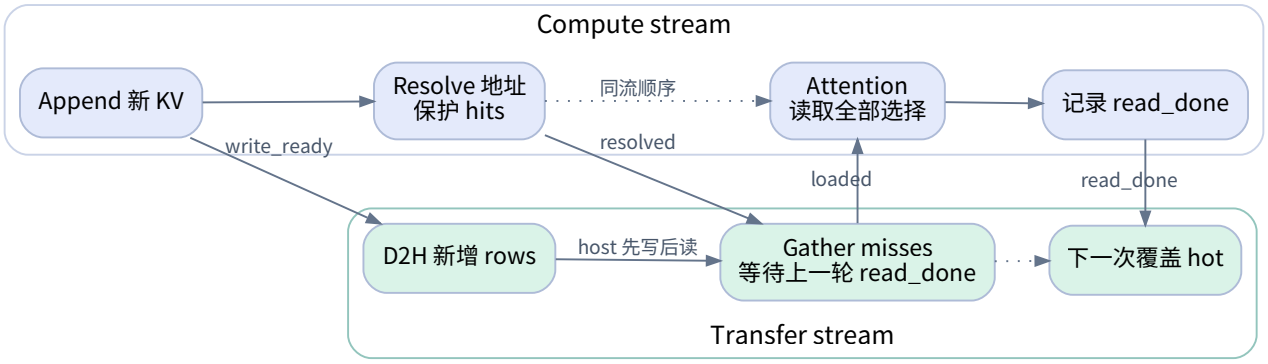


图5：同层 attention 必须等 miss gather；CPU release 记录的事件可能尚未完成，但下次覆盖必须等待它。图只表达依赖，不宣称已测得 overlap。

表项与数据有不同的就绪条件：resolver 可以先更新 tags 和落点，消费者仍必须等待最新 `hot_ready`，否则会读到旧 slot 内容。Miss-plan tensors 在 transfer 使用完前也必须存活；不能让异步分配器过早复用它们的内存。

基础实现选择一个 transfer stream，让初始 staging/后续 D2H 镜像与 host gather 有明确顺序。若拆成多个 transfer streams，就必须显式增加每个 source 页的 ready 依赖，不能再依赖“同流先写后读”。

7. 效率：减少搬运，并明确哪些等待可以隐藏

7.1 基础方案已经减少的工作

Resident/hot hits 不搬运，未选 KV 不加载，跨头重复 rows 只加载一次。若本步有 M 个唯一 misses，H2D payload 为 `M*2*kv_heads*head_dim*dtype_bytes`。镜像仅写新增 rows，并提前安排到真正回收之前。

固定形状 miss plan 可由设备直接消费，避免逐 miss 的 CPU 同步。Gather 写入最终 hot 落点，attention 随后直接按物理 row 读取。Pinned host 支持异步访问；Triton gather 可以由 SM 读取 host 的 CUDA 地址，**这不是 copy-engine DMA，仍可能与 attention 竞争 SM/带宽**。

这些手段降低搬运需求和控制往返，但基础流程仍是 `resolve → gather → attention`。同层缺失数据的等待仍在关键路径上，不能把使用两条 stream 当作充分隐藏延迟的证据。

7.2 进阶方案：先计算已经就绪的部分

将逐头 selection 拆成 `S_ready = resident hits ∪ hot hits` 与 `S_miss`。Resolver 保护 hits 后，两部分访问的 row 落点互不冲突，可以在加载 misses 时计算 ready 部分，最后合并 attention 统计量。

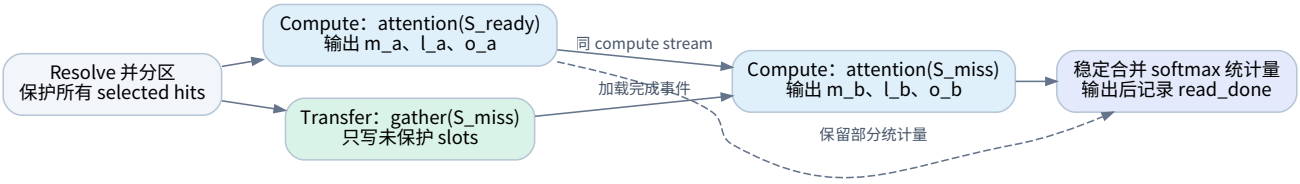


图6是进一步优化方案，尚未在最小实现中启用。整个 lease 要保持到两部分合并结束；gather 只能覆盖未保护 slots。

每个 head、每个分区返回 FP32 统计量：最大 score `m`、`l=Σexp(score-m)`、`o=Σexp(score-m)*V`。设两个分区为 a、b：

```
m = max(m_a, m_b)
α = exp(m_a - m), β = exp(m_b - m)
l = α*l_a + β*l_b
o = α*o_a + β*o_b
output = o / l
```

不能简单平均两次 attention 输出。空分区直接跳过，以免计算 `-∞-(-∞)`；当全部有效选择为空时返回零输出。分区只改变归约顺序，不应改变各 head 的选择集合。

忽略公共 resolver 开销，理想时间由 `T_gather+T_attention(all)` 变为约 `max(T_gather,T_ready)+T_miss+T_merge`。这是分析模型，不是实测结论；额外 kernel、归约与 SM 竞争可能抵消收益。应保留单次 attention 路径，基于 profiler 和同 selection 对照决定是否开启。

跨层预取的条件。 只有后续层的 index 已知、source 历史 rows 已就绪、该层 hot lease 有效时才可提前加载。不能假设下一 token 的 query/top-k 已知，也不能预取尚未产生的当前 token KV。多卡、共享 indexer 和深度流水需要另外设计，不属于基础进展保证。

8. 正确性与进展：必须始终成立的约束

不变量	防止什么问题
每个可读历史 row 有已完成的 host backing，或仍有受保护的 resident 原件	数据在镜像完成前被丢失
实际覆盖前撤销旧映射，并尊重旧读写 fence	旧请求读到新 owner 的数据
Resident 页或 hot slot 在消费者完成前不能被破坏	Attention 读写竞争
所有 selected hot hits 先保护，再分配 misses	后遇到的 hit 被先遇到的 miss 覆盖
每请求的 U、hot/tail credit、host/index credit 不被超订	已准入请求缺少基本工作集
选择集合与逐头 mask 不因缓存布局变化	通过漏掉 miss 获得错误的“加速”

进展推导。 对已准入请求，追加只依赖受保护的 tail；跨页后更老历史页可在镜像完成时让位。选择并集不超过 U，且 H≥U，所以保护 hits 后剩余 slots 足以容纳全部 misses。Source backing 已经存在或由有序事件确保完成，gather 最终结束，attention 得到完整选择集合并产生下一个 token。该过程不需要完整历史重新驻留。

这不是无限制无等待保证：读者不释放、设备失败、外部消耗掉权重/activation 所需显存等情况不在上述前提内。资源等待必须可观察并有明确唤醒条件，不能用忙等循环掩盖无候选的状态。

异常或压力场景	处理方式
新请求的 hot/tail 或 host/index credit 不足	留在 waiting 队列，不先执行 prefill 再赌后续容量
历史页镜像未完成，暂时不能回收	等待相关完成事件；异步调度器可切换其他请求
Hot 被当前消费者占用	保留 demand，消费者完成后重试；不覆盖其内容
Selection 超过 U	明确拒绝或重新配置准入；不截断 selection
请求取消、ID 再利用	增加请求代次；在途 backing/metadata 延迟释放
原位置尚未复用却再次被选择	继续 resident hit；不因 unpin 提前制造 miss

为何不要求全局最优驱逐？ 题目的首要约束是安全和可前进，最低工作集可把它们与命中率优化分开。初版采用确定的候选条件与队列；之后再用请求公平性、复用概率或带宽成本优化 victim 顺序，但不能破坏上述不变量。

为何不把整个请求重新 swap-in？ 那会再次要求与完整上下文长度成比例的瞬时 GPU 空间，正好破坏“部分驻留也能 decode”的目标。固定 hot 工作集将单步最低容量与完整历史长度解耦；host 和索引成本仍随历史增长，必须独立约束。

9. 验证计划、已有证据与实现边界

设计是否成立，需要分别验证**数据正确、可以前进、性能值得**。三者不能用同一个 hit rate 或测试数量代替。

验证层次	方法与判据
存储正确性	冻结逐头 selection，对比完整 K/V 上同 selection 的 attention；覆盖 miss、重复输入、padding、碎片化 blocks
生命周期安全	人为延迟 D2H、gather 或旧 reader；确认完成前不提前复用，取消/ID 重用后无旧数据泄漏
最小工作集进展	H=U，每步全部 misses，连续跨页追加；tail 保持驻留，输出正确且不重做 prefill
资源边界	超额请求推迟准入；host/index 预算和 GPU pool 容量均可核对
性能与策略	同 trace、同 selection、同 GPU KV 容量比较，单独记录 H2D bytes、索引成本、层级耗时及分阶段时间线

最小实现中的已有证据

mini-hisparse 有 96 项 CPU/CUDA 测试通过。其中 `P=4、U=H=4、GPU blocks=3` 的最坏 miss 用例，在 32-token prefill 后连续追加 16 次、产生 64 次 misses，每步输出正确。Engine 用例检查不重做 prefill；固定 prompt 的 Qwen2.5 完整选择生成有 32 个 greedy tokens 与 Hugging Face 一致。

实测为 A100-SXM4-40GB、BF16、P=16、14 Q heads / 2 KV heads、D=64；Qwen2.5-0.5B-Instruct 的三个 LongBench 样本，各 988 tokens，四层、八步，共 96 events。每配置五次，耗时取 run medians 的 median。

选择 / 同组 KV blocks	布局	H2D MiB / 96 events	Median ms / layer
Quest 每头 4 页 / 384	整页参考	1.141	2.391
Quest 每头 4 页 / 384	Hybrid rows	1.805	2.784
离线 oracle 每头 16 tokens / 204	整页参考	18.062	8.833
离线 oracle 每头 16 tokens / 204	Hybrid rows	1.086	2.682

只在同组内比较。计时含串行 append、解析/加载、attention 与同步，排除选择器、prefill、JIT 和 oracle。最大 BF16 存储绝对误差分别为 0.001953125 / 0.0078125，通过 `atol=rtol=0.008` 校验。输入是 frozen dense-teacher 激活，不是原生 DSA、闭环质量或 serving 吞吐；未记录有效权重 revision，不保证重新下载后的字节复现。

固定 384 blocks 时，Quest 将 H 从 2U 降到 U，H2D 从 1.805 降到 1.329 MiB，耗时却从 2.781 升到 2.844 ms。**少搬不一定更快，2U 不是普适最优。**

落地范围

已实现的基础机制	仍需进一步实现或测量
共享池、镜像、clean 回收、两页 tail、hot credit、row LRU	动态 hot 配额、生产异步调度
设备多算子 resolver、SM gather、row attention	Fused resolver、原生 DSA/MLA
基础事件链、数值与延迟读者测试	第7节分区重叠、跨层预取、端到端测量